

심화 실습 평가2(리눅스 프로그래밍)

TCP를 이용한 원격 장치 제어 프로그램

윤찬규

Contents

1. 개요	1
1.1. 프로젝트 소개	1
1.2. 전체 동작 흐름	1
1.3. 실행 환경	1
1.3.1. 서버 환경	1
1.3.2. 클라이언트 환경	1
1.3.3. GPIO 핀 배치	1
2. 설정과 빌드	2
2.1. 환경 설정	2
2.2. 빌드 및 배포	2
2.3. 실행	2
3. 명령 프로토콜	2
3.1. 클라이언트 로컬 명령	2
3.2. 서버 명령 목록	2
3.3. 응답 형식	3
4. 프로그램 구조	3
4.1. common	3
4.1.1. app_config	3
4.1.2. command	3
4.2. client	3
4.3. server	3
4.3.1. main	3
4.3.2. socket	3
4.3.3. parsing	3
4.3.4. server_log	3
4.3.5. device	3
5. 장치 제어 계층	3
5.1. 플러그인 로딩	3
5.2. 워커와 동기화	4
5.3. LED	4
5.4. 부저	4
5.5. 조도센서	4
5.6. 7세그먼트	4
6. 빌드 시스템 구조	4
7. 문제점 및 개선 방안	4
8. 일정	4

1. 개요

1.1. 프로젝트 소개

TCP Remote Device Control System은 TCP 소켓 통신으로 Raspberry Pi에 연결된 GPIO 장치를 원격 제어하는 C 기반 프로그램입니다.

다. 사용자는 Ubuntu/Linux 클라이언트의 콘솔에서 텍스트 명령을 입력하고, 서버는 Raspberry Pi에서 데몬 프로세스로 동작하면서 LED, 부저, 조도센서, 7세그먼트 디스플레이를 제어합니다.

주요 특징

- TCP 기반 명령/응답 구조: 클라이언트가 한 줄 명령을 보내면 서버가 OK, DATA, ERR 형식의 응답을 반환합니다.
- epoll 기반 서버 루프: 서버는 epoll로 여러 클라이언트 연결과 수신 이벤트를 처리합니다.
- 장치 플러그인 구조: led, buzzer, light, segment 기능을 각각 공유 라이브러리(.so)로 빌드하고 런타임에 dlopen으로 로드합니다.
- 장치별 작업 스레드: LED, 부저, 조도센서는 전용 워커 큐에서 직렬 처리되며, 7세그먼트는 별도 갱신 스레드로 카운트다운을 수행합니다.
- 데몬 서버: 서버는 double fork와 setsid()를 이용해 백그라운드 서비스로 전환되고, 표준 출력과 오류는 log/server.log로 기록됩니다.
- 자동 빌드/배포: build.sh가 클라이언트 빌드, Raspberry Pi용 서버 크로스 컴파일, 서버 파일 전송을 자동화합니다.

1.2. 전체 동작 흐름

```

사용자 입력
↓
client/main.c
- poll()로 stdin과 서버 소켓을 동시에 감시
- 로컬 명령(.help, .clear, .quit)은 클라이언트에서 처리
- 서버 명령은 TCP로 전송
↓
server/socket.c
- epoll_wait()로 접속/수신 이벤트 처리
- 수신 버퍼를 exec_command()에 전달
↓
server/parsing.c + common/command.c
- 명령 파싱 및 값 검증
- device_manager API 호출
↓
server/device
- 플러그인 로드
- 장치별 워커/스레드에서 GPIO 제어
↓
서버 응답 + server.log 기록
  
```

1.3. 실행 환경

1.3.1. 서버 환경

- OS: Raspberry Pi OS 64-bit
- 대상 보드: Raspberry Pi 4 Model B 또는 GPIO 호환 보드
- GPIO 라이브러리: wiringPi, softPwm, softTone
- 런타임 파일: server, config.ini, libled.so, libbuzzer.so, liblight.so, libsegment.so, resource 디렉터리

1.3.2. 클라이언트 환경

- OS: Ubuntu 24.04 LTS 또는 GCC/CMake를 사용할 수 있는 Linux 배포판
- 컴파일러: C17 지원 GCC/Clang
- 빌드 도구: CMake 3.11 이상
- 네트워크: 서버의 TCP 포트에 접근 가능해야 합니다.

1.3.3. GPIO 핀 배치

코드는 wiringPiSetupGpio()를 사용하므로 아래 번호는 BCM GPIO 번호입니다.

장치	GPIO 핀
LED	17
부저	18
조도센서	27
7세그먼트	A=5, B=6, C=13, D=19, E=26, F=20, G=21, DP=16

2. 설정과 빌드

2.1. 환경 설정

프로젝트 루트의 config.ini는 클라이언트 접속 정보와 Raspberry Pi 배포 정보를 함께 담습니다. 현재 C 코드의 load_app_config()는 [Server] 섹션의 addr, port를 읽고, build.sh는 파일 전체의 RPI_USER, RPI_IP, RPI_DEST 값을 셸 변수로 읽어 서버 배포에 사용합니다.

```
[Server]
addr=100.93.201.42
# addr=172.31.235.152
port=8080
backlog=10

[RaspberryPi]
RPI_USER=dldyou
RPI_IP=100.93.201.42
RPI_DEST=~/.TCPRemoteDeviceControlSystem
```

서버는 addr 값으로 바인딩하지 않고 INADDR_ANY로 모든 인터페이스에서 연결을 받습니다. 클라이언트는 addr, port 값을 사용해 서버에 접속합니다.

2.2. 빌드 및 배포

프로젝트 루트에서 다음 명령을 실행합니다.

```
chmod +x ./build.sh
./build.sh
```

build.sh의 실제 수행 단계는 다음과 같습니다.

- build_client에서 클라이언트를 로컬 GCC로 빌드합니다.
- build_server에서 raspberrypi4_toolchain.cmake를 사용해 서버를 Raspberry Pi 4 대상으로 크로스 컴파일합니다.
- server, 네 개의 플러그인 공유 라이브러리, resource/*, config.ini를 임시 디렉터리에 모읍니다.
- scp로 \${RPI_USER}@\${RPI_IP}:\${RPI_DEST}에 전송합니다.

빌드 산출물은 다음 구조로 배포됩니다.

```
${RPI_DEST}/
├─ server
├─ config.ini
├─ libled.so
├─ libbuzzer.so
├─ liblight.so
├─ libsegment.so
└─ *.wav
```

2.3. 실행

서버 실행

```
cd ~/.TCPRemoteDeviceControlSystem
./server
```

서버는 실행 직후 데몬으로 전환됩니다. 이후 로그는 실행 파일이 있는 디렉터리 기준 log/server.log에 기록됩니다.

클라이언트 실행

```
cd /path/to/TCPRemoteDeviceControlSystem
./build_client/client/client
```

클라이언트가 서버에 연결되면 device> 프롬프트가 표시됩니다.

3. 명령 프로토콜

3.1. 클라이언트 로컬 명령

아래 명령은 서버로 전송되지 않고 클라이언트에서 직접 처리됩니다.

- .help 또는 client help: 로컬 도움말과 서버 명령 목록 출력
- .clear 또는 clear: 터미널 화면 지우기
- .quit, .exit, exit: 클라이언트만 종료
- quit: 서버에 QUIT을 전송한 뒤 클라이언트 종료

클라이언트는 poll()로 표준 입력과 서버 소켓을 함께 감시하므로, 사용자가 입력하지 않는 동안에도 서버 응답을 즉시 출력할 수 있습니다.

3.2. 서버 명령 목록

명령 파서는 대소문자를 구분하지 않으며 공백으로 토큰을 분리합니다.

명령	동작
HELP	사용 가능한 서버 명령을 반환합니다.
STATUS	초기화 여부, LED/부저 상태, 마지막 조도 값, 7세그먼트 상태를 반환합니다.
PLUGIN LIST	LED, 부저, 조도센서, 7세그먼트 플러그인 로드 상태를 반환합니다.
DEVICE LIST	PLUGIN LIST와 같은 동작을 수행합니다.
LED ON	LED를 켭니다.
LED BRIGHT <0-255>	PWM 밝기 값으로 LED를 켭니다. 값이 0이면 상태는 꺼짐으로 저장됩니다.
LED OFF	LED를 끕니다.
BUZZER ON	기본 톤 440Hz로 부저를 켭니다.
BUZZER OFF	부저를 끄고 대기 중인 부저 작업을 취소합니다.
BUZZER BEEP <1-10000>	지정한 밀리초 동안 기본 톤으로 부저를 울립니다.
BUZZER MUSIC <file>	PCM WAV 파일을 읽고 피치를 감지해 부저로 재생합니다.
LIGHT READ	조도센서 값을 읽고 값이 0이면 LED ON, 1이면 LED OFF를 수행합니다.
SEG DISPLAY <0-9>	7세그먼트에 숫자를 표시하고 1초마다 감소하는 카운트다운을 시작합니다.
SEG CLEAR	7세그먼트 표시를 지웁니다.

QUIT	OK QUIT 응답 후 서버에 SIGTERM을 발생시켜 서비스 루프를 종료합니다.
------	---

3.3. 응답 형식

서버 응답은 한 줄 문자열입니다. 클라이언트는 OK로 시작하면 초록색, ERR로 시작하면 빨간색으로 표시합니다.

```
OK LED ON
OK BUZZER BEEP 500
DATA LIGHT 0 LED=ON
DATA STATUS INIT=ON LED=ON BUZZER=OFF LIGHT=0 SEG=ON
SEG_VALUE=3
DATA PLUGINS LED=LOADED BUZZER=LOADED LIGHT=LOADED
SEGMENT=LOADED
ERR INVALID_COMMAND
ERR INVALID_VALUE
ERR DEVICE_ERROR PLUGIN_LOAD_FAILED
```

4. 프로그램 구조

4.1. common

4.1.1. app_config

app_config는 서버 주소와 포트를 담은 전역 설정 구조체를 제공합니다. 기본값은 addr=127.0.0.1, port=60000이고, config.ini의 [Server] 섹션에서 addr, port를 덮어씁니다.

4.1.2. command

command_parse()는 입력 문자열을 ParsedCommand로 변환합니다. 명령 길이는 256자 미만이어야 하며, 정수 인자는 지정된 범위 안에 있어야 합니다. 문자열 인자는 현재 한 토큰만 허용되므로 BUZZER MUSIC 파일 경로에는 공백을 넣을 수 없습니다.

4.2. client

클라이언트는 config.ini를 읽고 TCP 소켓으로 서버에 연결합니다. 콘솔 루프는 poll() 기반으로 동작하며, 사용자 입력과 서버 응답을 동시에 처리합니다.

주요 함수는 다음과 같습니다.

- set_signal_handler(): SIGINT만 종료 신호로 처리하고 SIGPIPE, SIGTERM, SIGHUP, SIGQUIT, SIGTSTP는 무시합니다.
- handle_local_command(): 로컬 UI 명령과 종료 명령을 처리합니다.
- send_command(): 입력 명령 뒤에 개행을 붙여 서버로 전송합니다.
- print_server_response(): 응답 접두어에 따라 색상을 적용합니다.
- run_console(): 클라이언트의 메인 입력/수신 루프입니다.

4.3. server

4.3.1. main

서버는 실행 파일 위치를 기준으로 작업 디렉터리와 config.ini 경로를 결정합니다. 설정을 읽은 뒤 데몬으로 전환하고, 장치 플러그인 초기화가 성공하면 소켓을 열어 서비스 루프를 시작합니다.

데몬화 과정은 다음 순서로 진행됩니다.

- fork() 후 부모 프로세스 종료
- setsid()로 새 세션 생성
- SIGHUP 무시 후 두 번째 fork()
- umask(0) 설정 및 실행 파일 디렉터리로 chdir()
- 기존 파일 디스크립터 닫기

- stdin은 /dev/null, stdout/stderr는 log/server.log로 리다이렉트

4.3.2. socket

socket_init()은 TCP 소켓을 생성하고 SO_REUSEADDR를 설정한 뒤 INADDR_ANY:app.port로 바인딩합니다. service()는 서버 소켓과 클라이언트 소켓을 epoll에 등록해 접속과 명령 수신을 처리합니다.

명령을 수신하면 다음 순서로 처리합니다.

- 원본 요청을 복사합니다.
- exec_command()가 수신 버퍼를 서버 응답으로 변경합니다.
- log_client_command()가 시간, 클라이언트 주소, 요청, 응답을 기록합니다.
- 해당 클라이언트에게 응답을 전송합니다.

4.3.3. parsing

exec_command()는 command_parse() 결과에 따라 장치 관리자 함수를 호출합니다. 유효하지 않은 명령은 ERR_INVALID_COMMAND, 값 범위 오류는 ERR_INVALID_VALUE, 장치 계층 오류는 ERR_DEVICE_ERROR <원인>으로 정규화합니다.

4.3.4. server_log

서버 로그는 한 줄 단위로 시간, 클라이언트 IP/포트, 요청, 응답을 남깁니다.

```
[2026-06-05 10:30:12] 100.93.201.10:54231 "LED ON" -> "OK LED ON"
```

4.3.5. device

장치 제어 계층은 플러그인 로딩, 명령 실행, 상태 관리, 동기화 등을 담당합니다. 각 장치는 device_manager API로 추상화되어 있으며, 플러그인은 이 API를 구현하는 공유 라이브러리로 빌드됩니다.

5. 장치 제어 계층

5.1. 플러그인 로딩

서버는 plugin_registry_load()에서 네 개 플러그인을 모두 로드합니다. 하나라도 실패하면 초기화가 실패합니다.

플러그인	필수 심볼
libled.so	led_init, led_cleanup, led_on, led_on_bright, led_off, led_is_on
libbuzzer.so	buzzer_init, buzzer_cleanup, buzzer_music, buzzer_on, buzzer_off, buzzer_beep, buzzer_is_on
liblight.so	light_init, light_cleanup, light_read, light_get_last_value
libsegment.so	segment_init, segment_cleanup, segment_display, segment_clear, segment_is_enabled, segment_get_value

plugin_loader_open()의 탐색 순서는 다음과 같습니다.

- PLUGIN_LIB_DIR 아래의 lib{name}.so 또는 {name}/lib{name}.so
- DEVICE_LIB_DIR 아래의 같은 구조
- 서버 실행 파일 디렉터리의 lib{name}.so
- 서버 실행 파일 디렉터리의 device/{name}/lib{name}.so
- 현재 작업 디렉터리의 lib{name}.so

- 현재 작업 디렉터리의 device/{name}/lib{name}.so
- 동적 링커 기본 탐색 경로의 lib{name}.so

5.2. 워커와 동기화

LED, 부저, 조도센서는 각각 DeviceWorker를 하나씩 사용합니다. 워커는 내부 큐, mutex, condition variable을 가지고 있으며, 명령을 순서대로 처리합니다.

- wait=true: 호출자가 작업 완료까지 기다립니다. LED 제어, 부저 ON, 조도 읽기가 이에 해당합니다.
- wait=false: 큐에 넣은 뒤 즉시 성공을 반환합니다. 부저 음악 재생과 beep가 이에 해당합니다.
- device_worker_cancel_pending(): 아직 실행되지 않은 부저 작업을 취소할 때 사용합니다.

7세그먼트는 별도의 segment_refresh_loop() 스레드에서 1초마다 현재 값을 표시하고 감소시킵니다. 값이 0이 되면 표시를 지운 뒤 device_buzzer_beep(200)을 호출합니다.

5.3. LED

LED 플러그인은 GPIO 17을 출력으로 설정합니다. LED ON은 일반 GPIO 출력으로 HIGH를 쓰고, LED BRIGHT는 softPwmCreate()와 softPwmWrite()를 사용해 0-255 범위 밝기를 출력합니다. LED OFF는 PWM을 중지하고 LOW를 씁니다.

5.4. 부저

부저 플러그인은 GPIO 18에 softToneCreate()를 설정합니다. BUZZER ON과 BUZZER BEEP은 기본 톤 440Hz를 사용합니다. BUZZER OFF는 현재 출력과 음악 재생 루프를 중지하도록 요청합니다.

BUZZER MUSIC <file>은 PCM WAV 파일을 읽고 다음 방식으로 음을 재생합니다.

- WAV 헤더를 파싱하고 오디오 데이터를 mono signed 16-bit PCM 샘플로 변환합니다.
- 80ms 윈도우와 25ms hop 기준으로 샘플을 읽습니다.
- YIN 알고리즘으로 80Hz부터 2000Hz 범위의 피치를 감지합니다.
- 감지한 주파수를 가장 가까운 MIDI 음으로 양자화한 뒤 softToneWrite()로 출력합니다.
- BUZZER_PITCH_DEBUG 환경 변수가 0이 아닌 값이면 피치 추적 로그를 stderr에 출력합니다.

5.5. 조도센서

조도센서 플러그인은 GPIO 27을 입력으로 설정하고 digitalRead() 결과를 마지막 값으로 캐시합니다. 서버의 LIGHT READ 명령은 센서 값이 0이면 LED를 켜고, 1이면 LED를 끕니다. 응답에는 센서 값과 LED 제어 결과가 함께 포함됩니다.

5.6. 7세그먼트

7세그먼트 플러그인은 8개 핀을 출력으로 설정하고 숫자별 bit pattern을 GPIO에 씁니다. 현재 구현은 SEG_DISPLAY <0-9> 입력값을 카운트다운 시작값으로 사용하며, SEG_CLEAR로 표시와 카운트다운 상태를 초기화합니다.

6. 빌드 시스템 구조

최상위 CMake는 common 정적 라이브러리를 만들고, 옵션에 따라 client, server 하위 디렉터를 빌드합니다.

```
common
├── app_config.c
└── command.c

client
└── client 실행 파일

server
├── server 실행 파일
├── server_device 정적 라이브러리
└── led/buzzer/light/segment 공유 라이브러리
```

서버 실행 파일은 common, server_device, pthread, math, dl 라이브러리에 링크됩니다. 각 장치 플러그인은 SHARED 라이브러리로 빌드되며, Raspberry Pi에서는 wiringPi, crypt, Threads::Threads 등이 함께 링크됩니다.

7. 문제점 및 개선 방안

- BUZZER MUSIC 파일 경로는 공백 없는 한 토큰만 허용됩니다. 공백이 있는 경로를 지원하려면 문자열 인자 파싱 방식을 개선해야 합니다. 예를 들어, ""로 묶인 문자열을 하나의 인자로 처리하는 로직을 추가할 수 있습니다.
- 서버는 QUIT 명령이나 SIGTERM으로 종료 루프를 빠져나오지만, epoll_wait()가 다른 시그널로 중단되는 상황을 세밀하게 구분하지 않습니다. 안전 종료 처리를 더 명확히 다듬을 수 있습니다.
- 네 개 플러그인 중 하나라도 없으면 서버 초기화가 실패합니다. 장치 일부만 사용 가능한 모드를 지원하면 현장 테스트가 쉬워집니다.
- build.sh clean은 빌드 디렉터를 삭제하므로 실행 전에 산출물 보존 여부를 확인하는 안내를 추가하면 안전합니다.
- README의 일부 핀/파일 형식 설명은 현재 코드와 다를 수 있습니다. 문서와 README를 같은 기준으로 동기화하는 것이 좋습니다.

8. 일정

Day	Description
1일차	- 프로젝트 구조 설계 - CMake 기반 common, client, server 타킷 구성 - TCP 클라이언트/서버 기본 통신 구현 - 명령 enum과 공통 명령 파서 작성 - Raspberry Pi GPIO 장치 연결 및 기본 제어 테스트
2일차	- epoll 기반 서버 루프 구현 - 클라이언트 콘솔 UI와 로컬 명령 구현 - LED, 부저, 조도센서, 7세그먼트 플러그인 분리 - dlopen 기반 플러그인 로더와 registry 구현 - 장치별 워커 큐와 7세그먼트 카운트다운 스레드 구현 - 서버 데몬화 및 로그 시스템 구현
3일차	- WAV 파서와 YIN 기반 부저 음악 재생 기능 구현 - STATUS, PLUGIN LIST, SEG CLEAR, BUZZER BEEP 등 명령 보강 - Raspberry Pi 크로스 컴파일 및 자동 배포 스크립트 정리 - 최종 테스트와 문서 갱신